Project Report

Vision-Based Swarm Robotics for Autonomous Target Search and Tracking

Orobosa Igho-Osagie[a]

[a]*Blekinge Institute of Technology (BTH), Karlskrona, Sweden*

Compiled on June 2, 2026

Abstract

This paper presents the design and implementation of a simplistic PSO-inspired swarm robotics system where four Waveshare JetBot robots collaboratively search a dynamic environment for a randomly placed red box. Each robot relies exclusively on its onboard camera as the primary sensing modality, using HSV-based color detection to identify both the target and other members of the swarm by their distinctly colored flags. Distance to detected objects is estimated using the pinhole camera model, and directional control is achieved through a pixel-based differential steering strategy. A central system coordinates the swarm by periodically polling each robot for its latest detection status every 0.15, computing a fitness score defined as the estimated distance to the target, and broadcasting the identity of the global best robot to the remaining agents. Each robot autonomously executes a four-level priority hierarchy that governs whether it approaches the target directly, follows the global best robot, follows any visible robot, or continues searching. Experimental results demonstrate successful swarm convergence across multiple test scenarios, with all robots stopping within a defined proximity threshold of the target.

Keywords: *Swarm Robotics, Particle Swarm Optimization, JetBot, Object Detection, Color Detection, Autonomous Navigation, WiFi Network, Client–Server*

1. INTRODUCTION

With four robots placed at arbitrary positions in a room, our primary aim is to enable these robots to autonomously locate and track a red box that has also been randomly placed within a dynamic environment.

The environment is non-static, and this allows both the target and obstacles to move around unpredictably. As a result, all four robots must collaboratively search the environment and converge on the target. The robots used in this project are the Waveshare JetBots [2],which are equipped with onboard cameras and IMU sensors as their primary sensing devices. However, the robots do not possess GPS modules or any other sensors capable of providing precise positional information. Although the IMU data can be used to estimate instantaneous velocity and acceleration, the absence of a localization system prevents the robots from determining their exact positions within the environment.

For each robot, the primary source of input is the onboard camera feed. Since precise positional information is unavailable, determining the relative positions of the target and neighboring robots relies entirely on visual analysis. However, through communication with a central laptop over a shared Wi-Fi network, a search algorithm can combine the results of the visual information obtained from all four robots to enable the robots to operate as a coordinated swarm under centralized logic, allowing the problem space to be explored more efficiently and for the robots to find the target faster.

The centralized logic employed in this project is inspired by Particle Swarm Optimization (PSO) [1] originally proposed by James Kennedy and Russell Eberhart in 1995. In PSO, individual agents navigate a search space by balancing their own best-known position with the best position discovered by the swarm as a whole.

In our implementation, the fitness of each robot is determined by the estimated distance of the robot from the red box. At any point in time, the closest robot to the target is declared the 'global best'.

1.0.1. AI Usage

AI tools, specifically Claude (Anthropic), were used in the preparation of this report to assist with writing, proofreading, and refining technical explanations. All technical content, system design decisions, implementation, and experimental results are the work of the authors.

1.0.2. Report Organization

This report is organized as follows: **Section II** defines the problem formally; **Section III** describes our solution architecture and implementation; **Section IV** presents results and analysis; and **Section V** summarizes our findings and outlines the remaining work.

2. TASK DEFINITION: PROBLEM STATEMENT

With the above presented objective, our tasks were further defined as follows:

1. With the use of just the onboard camera, the robots must be able to detect both the target, and other robots in the environment for the swarm operation.
2. The robots must be able to navigate to and estimate their distance from the target once it has been detected.
3. Positional information relative to the red box must be collected by the central system from all four robots and used to determine the robot closest to the target.
4. All the above must then be combined to allow the robots to sweep the environment and converge at the target.

3. SOLUTION

3.1. Modeling and Conceptualization

Our solution consists of four phases that have now been called 'Three Ds and S', which stands for Detection, Distance, Direction and Swarm Operation.

3.1.1. Detection

As mentioned earlier, the primary source of input for each robot is the onboard camera feed. Therefore, in order for the robots to detect the red box and identify neighboring robots, a distinguishable visual feature was required, for which color was selected as the primary visual property for detection.

Object detection was then performed using OpenCV and simple HSV (Hue, Saturation, Value) thresholding. Each robot continuously

This report is typeset in LaTeX using the rho-class template [3].

captured camera frames and converted them from BGR to HSV color space. HSV was selected because it separates color information from brightness, allowing stable detection under changing lighting conditions. Distinct colors with colored flags were assigned to each robot (Blue, Purple, Yellow and Orange) and thus, in every frame, the robots searched for five colors which correspond to the colored flags and red box. To ensure robust thresholds that sufficiently captured the chosen colors, the color thresholds were supplied by Anthropic's Claude LLM and the utilized thresholds are listed below:

- **Red:**
  1. Mask 1: $H : [0, 10], S : [120, 255], V : [70, 255]$
  2. Mask 2: $H : [170, 180], S : [120, 255], V : [70, 255]$
- **Blue:** $H : [100, 130], S : [150, 255], V : [50, 255]$
- **Orange:** $H : [11, 18], S : [200, 255], V : [200, 255]$
- **Yellow:** $H : [20, 40], S : [100, 255], V : [100, 255]$
- **Purple:** $H : [130, 160], S : [50, 255], V : [50, 255]$

The red color has two different masks because red occurs on both ends of the hsv spectrum. These two masks allows red detection from both edges. After thresholding, contour detection was applied to locate connected regions of the target color. The largest valid contour of at least 50 pixels was selected and enclosed with a bounding box, providing the object's position and size within the image frame. 50 pixels was selected as threshold to allow the robots detect objects from approximately 80cm while ignoring tinier objects of a similar colors in the frame.

3.1.2. Distance Estimation

Once the target was detected, distance estimation was calculated using the pinhole camera model.

$$Est.\,distance = \frac{Real\ Size \times Focal\ length}{Pixel\ size} \tag{1}$$

To make use of this model, the focal length of the camera was experimentally calculated. This was done by placing the red box at a known distance and using the real world size and pixel dimension calculated from the contour box by the system, of the target to estimate the focal length in cm.

Rather than using just one dimension- the height or width of the box, the average of the height and width was taken, as the real size and pixel size. This was done to mitigate any estimation errors that might come from the target being placed in a different orientation than was used during calibration, so the robots viewing the box from a different angle.

The experiment was performed a few times and the median value of ***257.39 cm*** was selected as the estimated focal length of the cameras. This figure is then used as constant with the pinhole camera model to estimate the distance of the robots to the red box. Once the distance to the red box has been estimated, this value is stored in the local status dictionary contained on each robot's code. Information from this dictionary is utilized later on during the swarm operation.

3.1.3. Direction - Steering

Steering the robots toward a target (red box or other robots) depends entirely on pixel geometry. The robot uses the horizontal position of the detected target within the camera frame to determine its steering direction. Since the camera projection captures the relative direction of the target, no additional localization information is required.

After applying detecting a target and applying a bounding box, a steering error is calculated by measuring the horizontal difference between the center of detected object and the center of the image frame.

$$+VE\ error \rightarrow Target\ is\ to\ the\ RIGHT$$

$$-VE\ Error \rightarrow Target\ is\ to\ the\ LEFT$$

This steering error is then normalized to the range [-1,1] and used to set the motor commands. Larger errors result in stronger turning corrections and reduced forward speed, while smaller errors allow the robot to maintain a higher forward speed with only minor steering adjustments. This method allows for a smooth, continuously curved trajectory toward the target, rather than a stop-turn-drive behavior. The resulting motor commands are continuously applied as differential speed pairs to the left and right wheels, with a minimum speed clamp enforced to ensure that both wheels continue moving forward during target approach.

```
def steer_toward(error_x):
    normalized = max(-1.0, min(1.0, error_x / 240.0))
    forward    = BASE_SPEED * (1 - abs(normalized) * 0.5)
    turn       = normalized * 0.15
    robot.set_motors(max(0.05, forward + turn),
                     max(0.05, forward - turn))
```

Code 1. Steering function

A stop distance of 40cm was implemented and so, as the robot approaches the target, if the estimated distance calculated in an instance is less than 40cm, the stop command is initiated.

3.1.4. Swarm Operation

The swarm coordination system was inspired by the Particle Swarm Optimization (PSO) algorithm. Each robot independently searched for the red box while also working with shared information from a central system over WiFi.

Each robot has localized information on its detection data (if it has seen the red box and the estimated distance) that is updated every 0.15 seconds. The central system pools that information every 0.5 seconds from all four robots, to calculate which robot is closest to the red box. This robot is then crowned the 'global best'. In an attempt to mitigate potential issues that may occur from network latency, our system was designed such that all motion is executed locally by each robot, while the central server only pools positional information from all robots, get the global best and then broadcasts the current global best position at each time step. This structure ensures that the robots never block or wait for the network. The internal control loop runs continuously on each robot, using the most recently available global best information to decide on an action.

Given the range of movements available to the robot, its actions at every instant, or iteration of the list, are determined using a priority-based decision framework. The order of priorities is defined as follows:

1. **Priority One**: If a robot sees the red box, steer towards it.
2. **Priority Two**: Else - If the robot sees the flag of the robot closest to the red box, steer towards the global best robot.
3. **Priority Three**: Else - If the robot sights any other flag from the other robots, steer towards that robot
4. **Priority Four**: Else - Spin and Search

Everytime this decision loop is iterated, the available option of the highest priority is executed, and so even if Robot B if following A, if A spots the red box, it steers towards it.

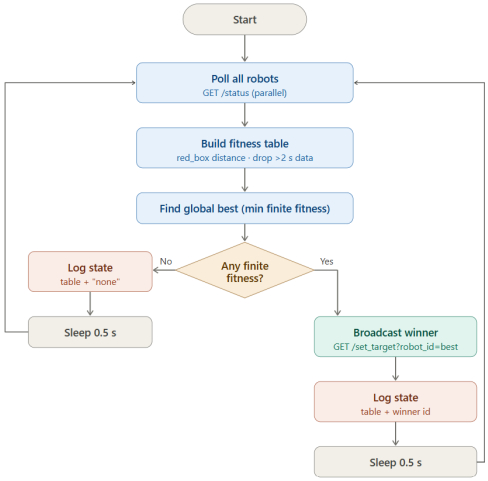

Figure 1. Central System Flowchart.

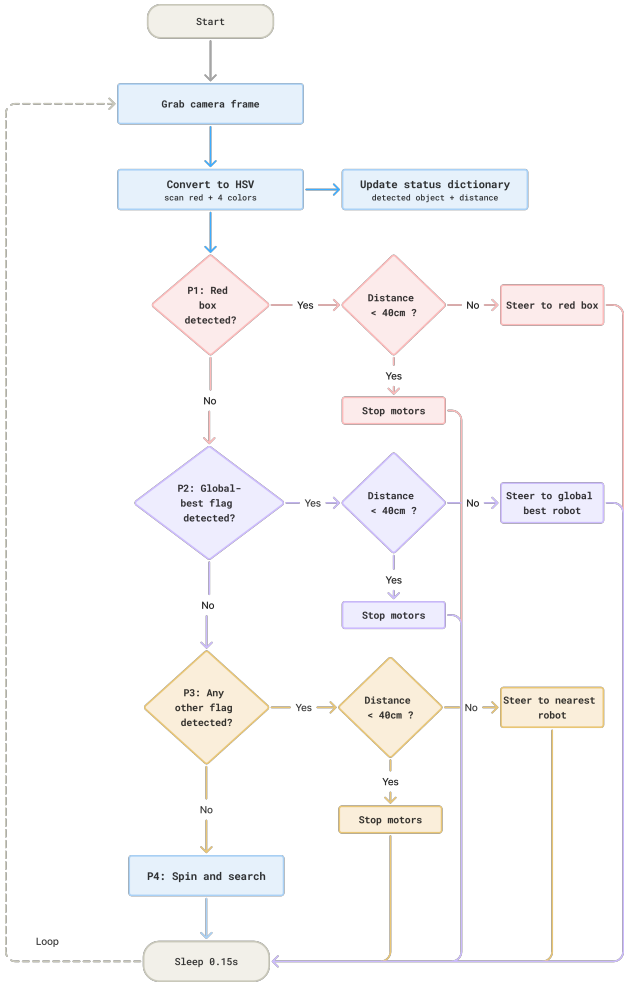

Figure 2. Robot Loop Flowchart.

3.2. Implementation

3.2.1. Network Architecture

The system utilizes a decoupled, asynchronous client–server architecture over a shared local Wi-Fi network. As described earlier, robot control and swarm coordination are intentionally separated, to allow each robot operate independently while periodically receiving updates from the central system. This design was chosen to prevent network latency from interrupting motion execution and to improve the robustness of the overall system.

Each JetBot runs a lightweight HTTP server (`Python http.server`) on port 8089. The server exposes a set of REST-style endpoints for motor control (`/set_motors`, `/forward`, `/left`, `/right`, `/stop`), camera image retrieval (`/camera`), detection status reporting (`/status`), and coordination updates (`/set_target`). The `/status` endpoint returns the robot's most recent detection results as a JSON object, which is retrieved by the central laptop at 0.5-second intervals to construct the PSO fitness table. The `/set_target` endpoint receives the identifier of the current global best robot from the laptop, which is stored locally and used within the robot's priority-based decision logic. Note that every robot had hard-coded within their codes the ID and color of all the robots in the swarm. This manually entered during the course of this experiment.

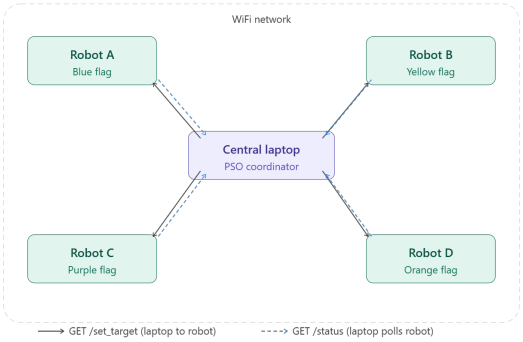

Figure 3. Centralized Network Architecture.

3.2.2. The Robot Loop

To make all the robots begin motion at the same time, the central system sends a `/start` command concurrently to all four robots, and this command initiates the control loop of the robots. Each iteration of this loop begins by capturing a frame directly from the onboard camera via `camera.value.copy()`. This frame is then passed to the `scan_frame()` function, which performs five independent color detection passes and returns a dictionary of bounding boxes for all detected targets.

This dictionary is subsequently processed by the `update_status()` function, which computes the distance to each detected target using the pinhole camera model and updates the `latest_detection` dictionary within the robot. The `red_box` information is accessed during the next polling cycle from the central laptop to build the fitness table. All other distance information is used by the robot to check if the stop distance to the robot it is following has been reached.

Next, `decide_and_act()` evaluates the current priority hierarchy against the latest detection results and issues the appropriate motor command. The loop then concludes with a 150 ms sleep interval, after which the cycle repeats. Since motor commands persist until explicitly overwritten, the robot continues executing the last issued command during the sleep period.

```
def main_loop():
    try:
        while True:
            frame        = camera.value.copy()
            scan_results = scan_frame(frame)
            update_status(scan_results)
            action = decide_and_act(scan_results, frame.shape[1])
            print(f"[{ROBOT_ID}] {action}")
            time.sleep(LOOP_DELAY)

    except KeyboardInterrupt:
        robot.stop()
        print("Stopped.")
```

Code 2. The main execution loop in the robots

3.2.3. The Central system loop

Each iteration of the central control loop begins by polling all robots simultaneously via parallel threads, each issuing a `GET /status` request to its respective robot and storing the JSON response in the shared `robot_states` dictionary. Once all threads complete, the loop computes a fitness score for each robot, set as its reported distance to the red box if detected and fresh, or infinity if no detection was reported or if the last response is older than two seconds. The robot with the smallest finite fitness is designated the global best, and its identifier is broadcast to all robots via `GET /set_target`, again using parallel threads. If all fitness scores are infinity, no broadcast is issued and the robots continue their current behavior. The loop concludes with a 500ms sleep before the next polling cycle begins.

```
try:
    while True:
        # Step 1 - poll all robots in parallel
        poll_all()

        # Step 2 - find global best
        global_best = find_global_best()

        # Step 3 - log fitness table
        print("\n--- Fitness Table ---")
        for robot_id in ROBOT_IPS:
            fitness = get_fitness(robot_id)
            label   = f"{fitness:.1f}cm" if fitness < float('inf'
    ↪ ) else "no detection"
            print(f"  Robot {robot_id}: {label}")
        print(f"  Global best: {global_best}")

        # Step 4 - broadcast global best to all robots
        if global_best:
            for robot_id in ROBOT_IPS:
                threading.Thread(
                    target=set_target,
                    args=(robot_id, global_best),
                    daemon=True
                ).start()

        time.sleep(POLL_INTERVAL)

except KeyboardInterrupt:
    print("Central system stopped.")
```

Code 3. Central System Logic

3.2.4. Challenges Faced During Implementation

These are a few bottlenecks faced while working on this project with how they were solved.

1. When two robots both activated priority level 3 of the decision logic and approached each other head-on, they often continued driving into each other and this caused head-on collisions, which was a frequent failure case during testing. To mitigate this issue, we modified the physical setup of the robots by completely occluding the front-facing portion of the colored flags using white material. This made sure that the color-based detection system primarily recognized the flags when viewed from behind the robots, reducing mutual detection in head-on encounters.

2. The HSV color boundaries, the detection system was generally robust at recognizing different shades of the required colors. However, during experimentation, the orange flag initially used on one of the robots appeared closer to brown in the camera images. To compensate for this, the orange threshold range was widened to include these darker shades. Although this improved flag detection, it introduced a new problem, in which brown shelves within the environment began to be falsely detected as the orange robot. The final solution was to tighten the threshold back to a strictly orange range and replace the original flag with a brighter orange marker.

4. RESULTS AND ANALYSIS

4.1. Live Demos

The following are links to videos with the live demonstration of the system in operation:

- Demo 1: Best case scenario
- Demo 2: Second case scenario
- Demo 3: Third case scenario
- Demo 4: Fourth case scenario

4.2. Current Limitations

- **Experimental Setup**: With the current structure of the design, replicating the experiment requires a controlled environment to work efficiently. This translates to removing every object in the environment of a color that the robots are detecting (red, blue, orange, yellow, and purple). Because detection of these colors in any frame signals to the robot that the red box or another robot has been sighted, this could create false positives that could potentially disrupt the swarm operation.

- **Working Scenarios**: For the implemented strategy to work, at least one robot must have sighted the red box before meaningful swarm coordination can begin. Prior to any detection, the global best identifier remains undefined and no coordination broadcast is issued by the central laptop, and so each robot is left to operate solely on local visual information. With no red box sighted and no current global best, priority 3 is executed, making all the robots cluster to a point. This means the initial search phase is entirely undirected, and the time taken to achieve first detection is dependent on the starting orientations and positions of the robots relative to the box.

- **Colliding Paths**: With the current structure, the literal number one priority for all the robots in the swarm is to locate the red box and move towards it. This means that is robot B was initially following A towards the target, the moment B moves and sights the red box, priority one is triggered and both robots would end up colliding with each other. Increasing the stop distance from 20cm to 40cm was an attempt to mitigate this issue, but it didn't stop the side-to-side collision.

- **Obstacle avoidance**: In our current solution, no dedicated obstacle detection was implemented. This led to the robots occasionally colliding with chairs and the tables.

5. SUMMARY AND CONCLUSION

This project demonstrated the feasibility of a camera-only, PSO-inspired swarm system in which four JetBot robots successfully collaborated to locate and converge on a randomly placed target using visual information alone. The combination of HSV-based color detection, pinhole distance estimation, pixel-based differential steering, and a distributed priority hierarchy, though a very simple solution, was sufficient to produce consistent swarm convergence across different test scenarios.

Several directions exist for improving the robustness and generality of the system. Replacing the HSV thresholding approach with a learned object detector such as YOLO would significantly improve detection performance under varying lighting conditions and reduce sensitivity to color ambiguity between the target and other objects in the environment. The current exploration strategy, in which robots without visual input spin in place, could be improved by introducing

a random walk component, where robots rotate to a random heading and then advance a short distance before re-scanning. This would distribute the swarm more effectively across the search space, and avoid them converging at one spot when neither of them has sighted the red box.

Obstacle avoidance was initially explored but later removed due to interference with the stopping behavior. This remains an important extension for deployment in cluttered environments. Its reintroduction would require a more principled detection mechanism capable of distinguishing obstacles from targets at close range.

Despite these limitations, the system achieved its primary objective and provides a functional foundation upon which these extensions can be incrementally built and evaluated.

The complete source code for this project can be accessed via the following GitHub repository: https://github.com/OrobosaIO/RoboCode.

REFERENCES

[1] J. Kennedy and R. Eberhart, "Particle swarm optimization", in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, 1942–1948 vol.4. DOI: 10.1109/ICNN.1995.488968.

[2] Waveshare Electronics, *JetBot AI Kit – Product Wiki and Documentation*, Accessed: May 2026, 2024. [Online]. Available: https://www.waveshare.com/wiki/JetBot_AI_Kit.

[3] G. Jimenez, *Rho-class: A LaTeX2e document class for research articles and lab reports*, Version 3.0.0, 2026. [Online]. Available: https://github.com/MemoJimenez/Rho-class.